

Sequential Genetic Testing — Methodology

What We Tested, Against What, and Why

Rana Lab · June 2026

The Problem in One Paragraph

A family member tests positive for a hereditary cancer gene (e.g. BRCA1/2). The doctor now needs to decide **who else in the family to test, and in what order**. Each test costs money. Each result updates your beliefs about everyone else. The goal is to find carriers efficiently — minimising cost while maximising detection. This is a sequential decision problem. The exact optimal solution requires enumerating every possible belief state the doctor could be in, which explodes combinatorially: 9 people = 262,144 states; 15 people = over 1 billion. It is infeasible at clinical scale.

The Three Methods

- **Exact DP (Dynamic Programming)** — brute-force backward induction. Enumerates every belief state, computes the provably optimal policy. Used as *ground truth* to grade everything else. Breaks above ~9 family members.
- **ABCD-16 (lab’s existing model)** — Approximate Dynamic Programming with 16 hand-crafted features (carrier probabilities, pedigree topology, test margins). Solves a linear program instead of full DP. We replicated this to verify our exact DP solver is correct.
- **RL Agent (our contribution)** — a PPO-trained neural network that learns a testing policy by playing the game millions of times. Never enumerates states. Runs at constant speed regardless of family size.

The RL Model: PPO + MLP Actor-Critic

We use **PPO (Proximal Policy Optimization)** with an **MLP (Multi-Layer Perceptron) actor-critic** network.

What this means:

- **MLP** — a standard neural network. Takes the current belief state (carrier probabilities for each family member + their test status) as input, and outputs which person to test next, or to stop.
- **Actor-critic** — two heads on the same network: the *actor* picks the action, the *critic* estimates how good the current state is. The critic’s estimate is used to reduce variance during training.
- **PPO** — the training algorithm. Runs thousands of episodes, collects rewards and penalties, and nudges the network weights to make better decisions. Same algorithm used in OpenAI’s RLHF training for early ChatGPT.

Why PPO:

- **Stable** — has a built-in clipping mechanism that prevents the policy from changing too drastically in one update, avoiding catastrophic forgetting
- **Works for discrete action spaces** — our action space is discrete: test person 1, 2, 3...or stop. PPO handles this naturally
- **Battle-tested and easy to tune** — the default go-to for RL on sequential decision problems; reliable results without extensive hyperparameter search

Note: the codebase also has a **GNN (Graph Neural Network)** option that treats the family pedigree as a graph and could learn structural patterns across different family shapes. This was built but kept as secondary — not used in the main experiments.

Experiments Run

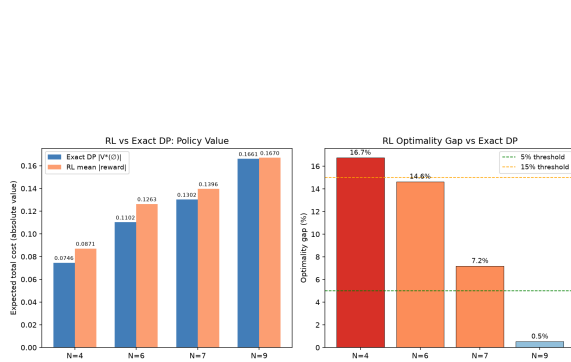
1 — RL vs Exact DP: Optimality Gap

Script: benchmark.py · Figure: fig2, fig7

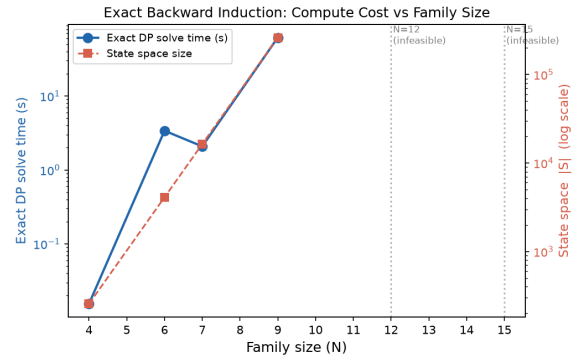
Trained RL separately on family sizes $N=4, 6, 7, 9$. Compared RL reward to exact DP reward to measure how far RL is from perfect. Also ran RL inference on $N=12, 15$ where exact DP cannot run at all.

Why: establishes whether RL is a credible approximation of the optimal policy.

Family size	Exact DP states	Exact DP time	RL gap from optimal	RL inference
$N=4$	256	0.015s	16.7%	0.12ms
$N=6$	4,096	3.4s	14.6%	0.12ms
$N=7$	16,384	2.1s	7.2%	0.13ms
$N=9$	262,144	61.8s	0.5%	0.12ms
$N=12$	16,777,216	infeasible	—	0.11ms
$N=15$	1,073,741,824	infeasible	—	0.12ms



(a) Optimality gap vs family size



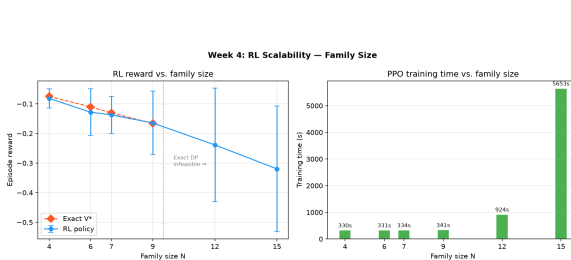
(b) Compute time: exact DP vs RL

2 — Scalability: Family Size and Gene Count

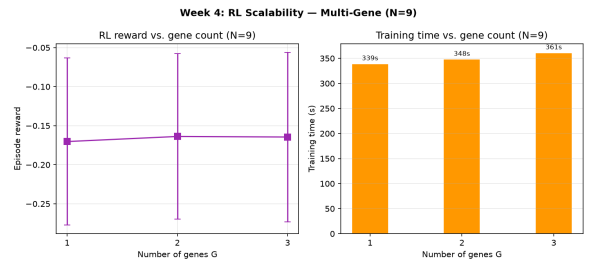
Script: benchmark_scale.py · Figure: fig3, fig5, fig6

Pushed family size from $N=4$ to $N=15$ and gene count from $G=1$ to $G=3$ on a fixed $N=9$ family. Exact DP is run where feasible; RL runs at all sizes.

Why: demonstrates that RL scales where exact DP breaks — the core practical argument.



(a) Scaling by family size



(b) Scaling by gene count

3 — Baseline Comparison: RL vs Random vs Myopic

Script: *eval_baselines.py* · Figure: *fig8*

Ran RL against three baselines on N=4, 6, 7, 9 over 500 evaluation episodes. Reward closer to 0 = better (less total penalty incurred).

Why: positions RL relative to simple strategies — random (floor) and myopic greedy (a strong heuristic that always tests the most suspicious person now, no forward planning).

Family size	Exact (perfect)	Random	Myopic greedy	RL
N=4	-0.0746	-0.1243	-0.0849	-0.0783
N=6	-0.1102	-0.1992	-0.1245	-0.1162
N=7	-0.1302	-0.2111	-0.1403	-0.1316
N=9	-0.1661	-0.2828	-0.1668	-0.1790

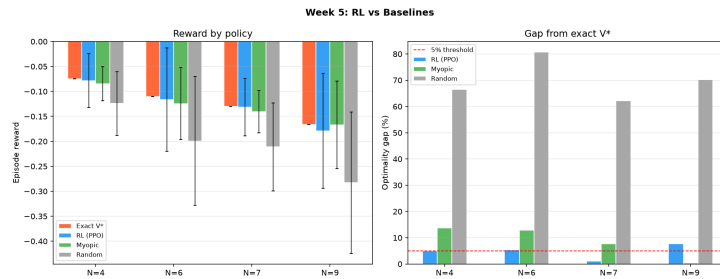


Figure 3: Baseline comparison across family sizes

4 — Sensitivity Analysis: Robustness Across Disease Prevalences

Script: *sensitivity.py* · Figure: *fig9*

Fixed N=9, varied allele frequency from 0.05 to 0.30 (covering rare to common hereditary mutations, including the BRCA1/2 range).

Why: checks whether RL is brittle — if it only works at one prevalence level, it is not clinically useful.

Result: RL gap from optimal stays between 0.5% and 9.3% across the full range. Robust.

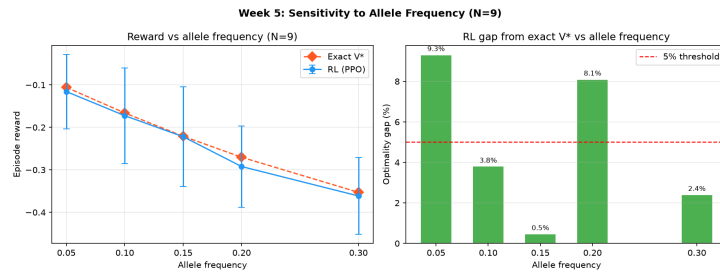


Figure 4: RL optimality gap across allele frequencies (N=9)

5 — Policy Behaviour: How Does RL Actually Decide?

Script: *visualize_policy.py* · Figure: *fig10*

Ran 2,000 episodes comparing RL vs myopic greedy on a 9-person family. Tracked number of tests run per episode and when each policy stops.

Why: reward numbers alone do not tell you *how* the policy behaves. This checks whether RL is doing something sensible — not just getting lucky.

Result: RL averages 6.05 tests per episode vs myopic’s 6.70. RL is more selective — it stops earlier without sacrificing detection quality.

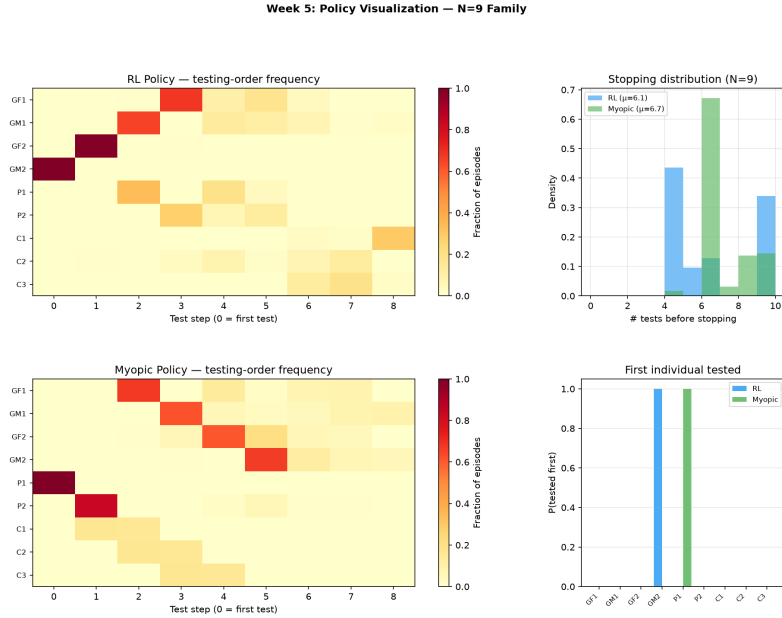


Figure 5: Policy behaviour: RL vs myopic greedy (2,000 episodes)

6 — ABCD-16 Replication: Validating Our Solver

Script: `replicate.py`

Re-ran the lab’s existing ABCD-16 model on 4 named families from the original report. Compared reproduced ratio2/ratio3 to published numbers.

Why: if our exact DP solver is wrong, every gap number above is meaningless. Replicating ABCD-16 to exact precision confirms the solver is correct.

Result: all 4 families reproduced with $\text{diff} = +0.00 \times 10^0$ — exact to every digit, far inside the 10^{-6} tolerance gate.

Family	ratio2 (policy gap)	ratio3 (certificate gap)
ThreeGeneration_LowHigh_Aggressive	5.1%	10.7%
Extended_LowHigh_Base	10.1%	26.8%
Extended_MediumEven_Base	15.9%	18.8%
Extended_LowHigh_Base (phase6)	42.3%	39.3%

What Is Still Missing

- **RL vs ABCD-16 head-to-head** — both methods were never run on the same families. Cannot currently claim RL beats ABCD-16 with data.
- **Train on a distribution of families** — RL currently trains separately per family size. A proper generalising agent trains on randomly sampled families and transfers.
- **Quality metric at N=12, 15** — RL runs at those sizes but we have no ground truth to grade it against (exact DP is infeasible). Need an alternative evaluation (e.g. compare to ABCD-16 or myopic).

- **Statistical significance** — results are over 500 episodes. Confidence intervals and significance tests needed before strong claims.